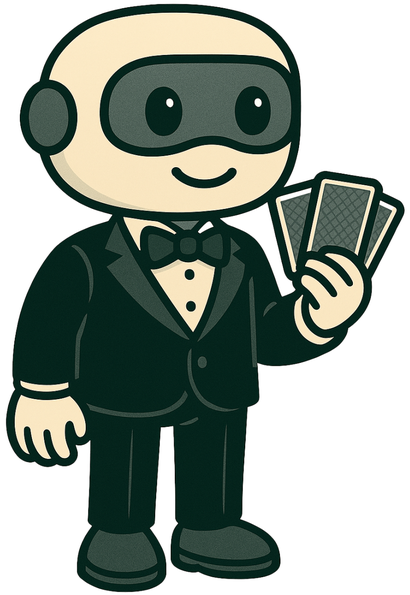
L8: Monte Carlo Methods

EECE 576N | From MDPs to Deep RL | Fall 2026

Cyrus Neary | cyrus.neary@ubc.ca



What is reinforcement learning?

How can we solve sequential decision-making problems without a model of the environment?

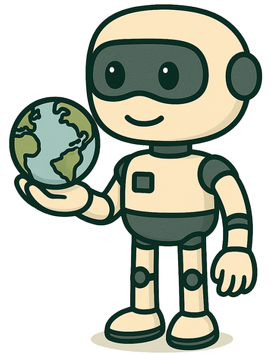
$$\mathscr{M} = (S, A, T, R, \gamma, \mu, \Omega, O)$$

? ? ? ? ? ?

Some or all of the (PO)MDP components are unknown.

The answer: learn from interactions with the environment.

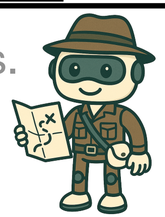
More specifically: Design an agent that gathers data from the environment and uses it to estimate π^*, $V^*(s)$, $T(s' \mid , s, a)$, $R(s, a)$, and other quantities.



How do we explore the unknown environment to gather useful experience for learning? How do we balance exploration with exploitation?

Exploration problems.

Last class.



How do we use our experience to update approximations of the value function and optimal policy?

Fundamental RL algorithms.

Next few lectures.

How do we build expressive function classes to help us learn and make decisions from diverse observation (e.g., images, language, etc.)?

Deep RL algorithms.

Part 3 of the course.

Last class: balancing exploration and exploitation

Exploration

- The agent doesn't initially know anything about the reward distributions $\mathscr{D}_a$.
- It needs to try out different actions to gather new information.
- If it incorrectly estimates action a_1 to have the highest mean reward, and thus always selects that action, how would it ever learn that another action has a higher mean reward?

Exploitation

- The agent's objective is to maximize cumulative reward.
- Given the information gathered in past rounds, it can make estimates of the mean rewards associated with each action.
- The agent should exploit that available information to try and obtain the highest possible reward in the near term.

This was all in the context of Bandits. We haven't yet talked about estimating rewards in general MDPs.

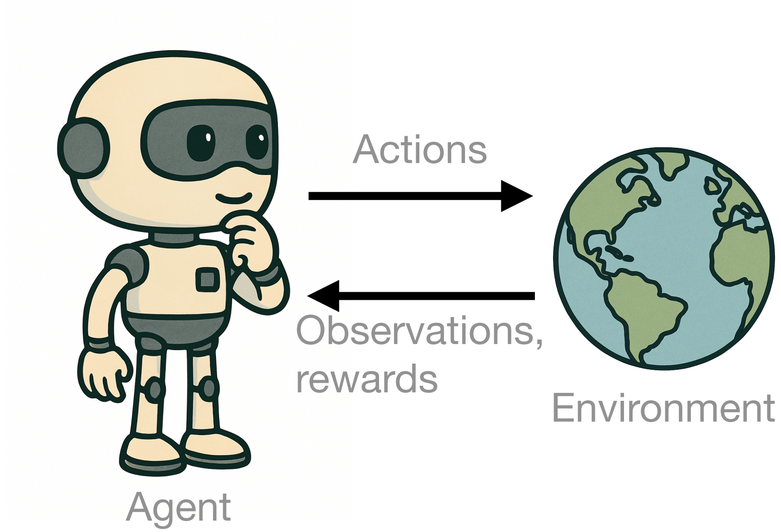
How do we design agents that intelligently balance exploration and exploitation throughout the decision-making rounds?

How do we evaluate the performance of a decision-making agent in this problem setting?

Bandit MDP:

Today: How can we learn value functions and policies from experience?

Consider an agent that can take actions in an environment, and as a result receives "experience" in the form of sequences of observations and rewards.



trajectory or episode

$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_T^1,$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_T^2$$

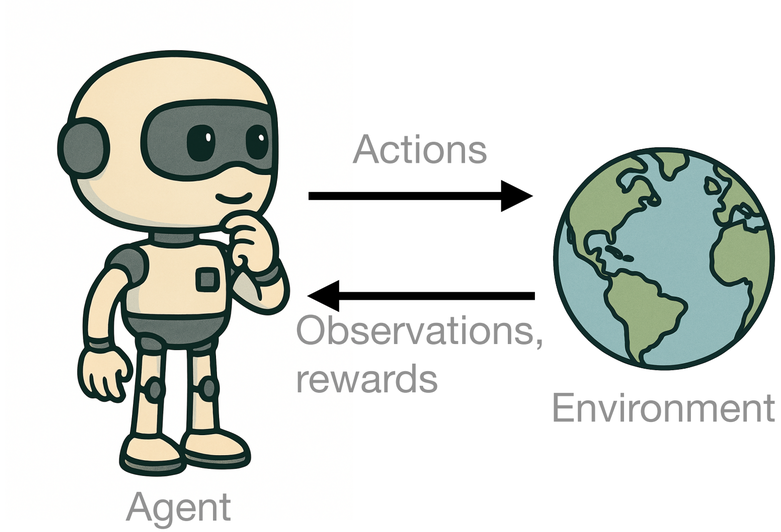
$$\vdots$$

episodes

initial state distribution μ

Today: How can we learn value functions and policies from experience?

Consider an agent that can take actions in an environment, and as a result receives "experience" in the form of sequences of observations and rewards.



$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_T^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_T^2$$

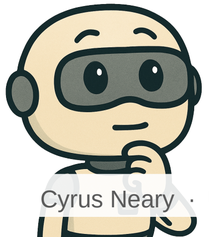
$$\vdots$$

When might we need to learn directly from experience in this way?

1. We don't know anything about the environment.

2. Computationally infeasible to fully solve DP problem. So instead, want a sampling-based approach.

3. We can simulate the environment, but it's challenging to write out $T(s'|s,a)$ exactly.



How can we use interactions, instead of an MDP model of the environment, to obtain an optimal policy?

what do we want to obtain from our sampled episodes?

↳ Learn $\hat{T}(s'|s,a)$ from data.

↳ Model-based RL

↳ Learn value function $\hat{V}(s)$, $\hat{Q}(s,a)$.

↳ Learn optimal policy directly.

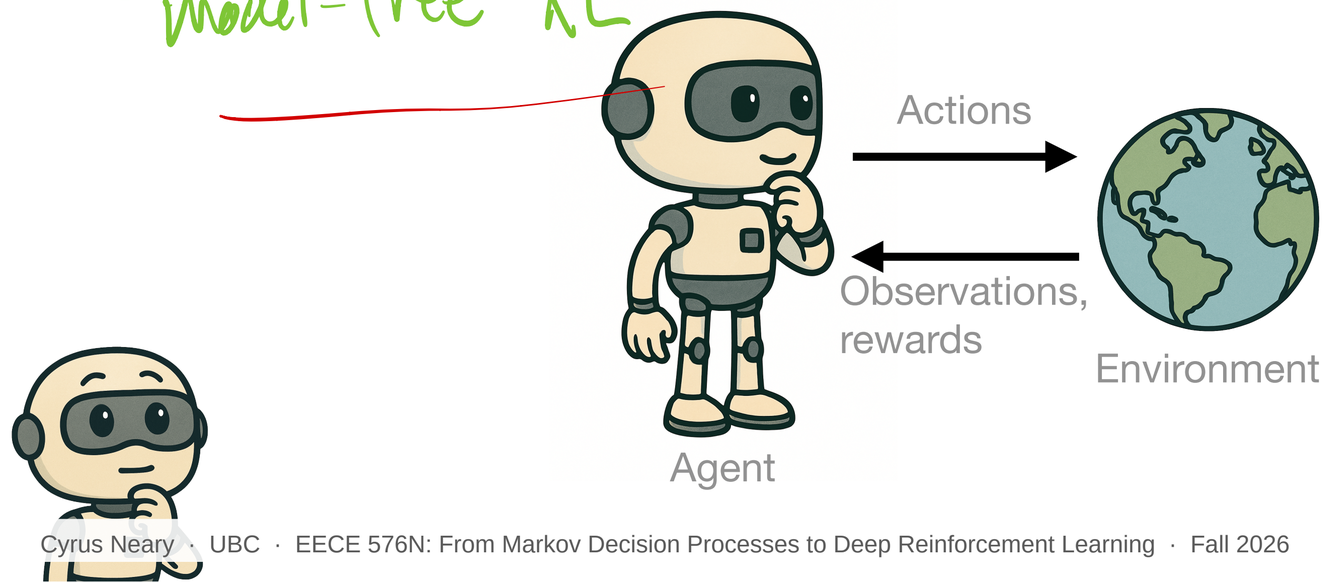
model-free RL

Once we have estimated the values, what do we do?

↳ policy ⟶ Done

↳ learned $\hat{V}$ or $\hat{Q}$

⟶ infer a policy by solving a one-step maximization problem.

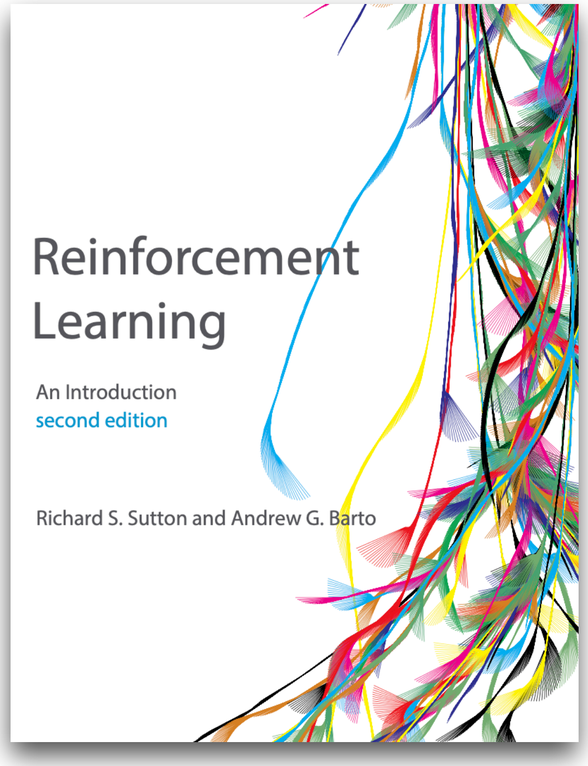


$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

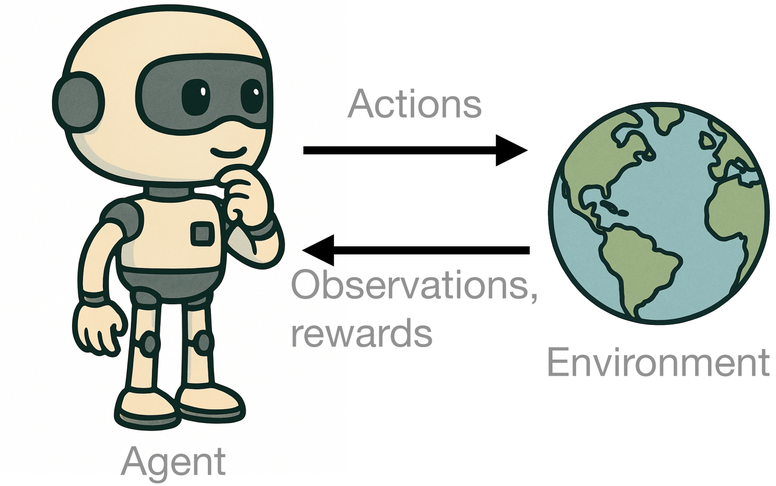
$$\vdots$$

Today: How can we learn value functions and policies from experience?



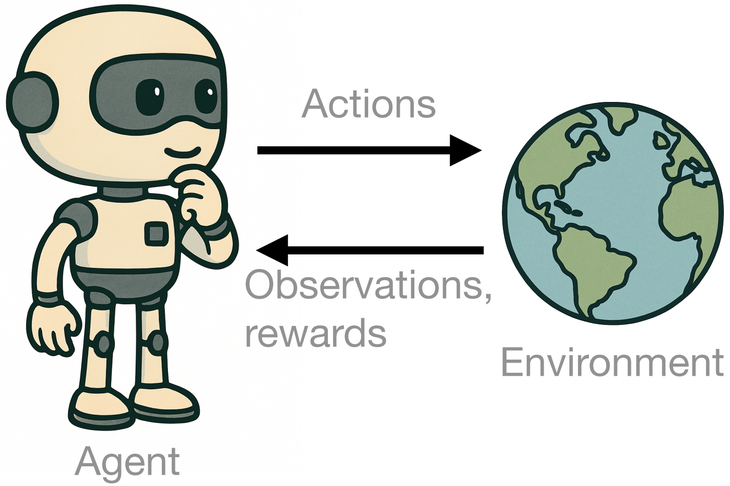
Chapter 5

Aside: Practically how do we implement RL algorithms?



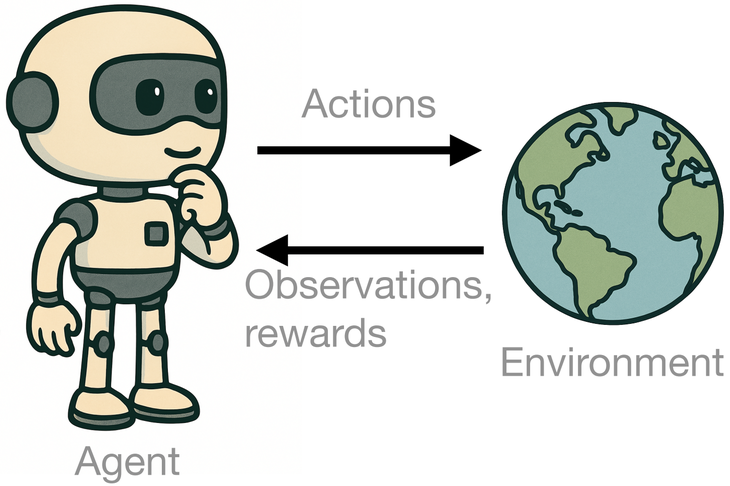
Aside: Practically how do we implement RL algorithms?

Our RL algorithms, implemented, e.g., in Python



Aside: Practically how do we implement RL algorithms?

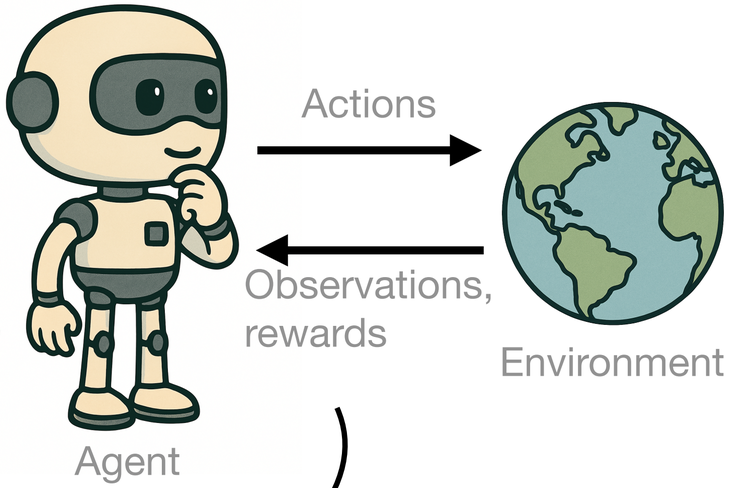
Our RL algorithms, implemented, e.g., in Python



Simulator (or real world)

Aside: Practically how do we implement RL algorithms?

Our RL algorithms, implemented, e.g., in Python



Simulator (or real world)

Simple, standardized API

Gymnasium

An API standard for reinforcement learning with a diverse collection of reference environments

Simple, standardized API

Gymnasium

class gymnasium.Env [source]

The main Gymnasium class for implementing Reinforcement Learning Agents environments.

The class encapsulates an environment with arbitrary behind-the-scenes dynamics through the `step()` and `reset()` functions. An environment can be partially or fully observed by single agents. For multi-agent environments, see PettingZoo.

The main API methods that users of this class need to know are:

- `step()` - Updates an environment with actions returning the next agent observation, the reward for taking that actions, if the environment has terminated or truncated due to the latest action and information from the environment about the step, i.e. metrics, debug info.
- `reset()` - Resets the environment to an initial state, required before calling step. Returns the first agent observation for an episode and information, i.e. metrics, debug info.
- `render()` - Renders the environments to help visualise what the agent see, examples modes are "human", "rgb_array", "ansi" for text.
- `close()` - Closes the environment, important when external software is used, i.e. pygame for rendering, databases

Environments have additional attributes for users to understand the implementation

- `action_space` - The Space object corresponding to valid actions, all valid actions should be contained within the space.
- `observation_space` - The Space object corresponding to valid observations, all valid observations should be contained within the space.
- `spec` - An environment spec that contains the information used to initialize the environment from `gymnasium.make()`
- `metadata` - The metadata of the environment, e.g. *{"render_modes": ["rgb_array", "human"], "render_fps": 30}*. For Jax or Torch, this can be indicated to users with *"jax"=True* or *"torch"=True*.
- `np_random` - The random number generator for the environment. This is automatically assigned during `super().reset(seed=seed)` and when assessing `np_random`.

Simple, standardized API

Gymnasium

class gymnasium.Env [source]

The main Gymnasium class for implementing Reinforcement Learning Agents environments.

The class encapsulates an environment with arbitrary behind-the-scenes dynamics through the `step()` and `reset()` functions. An environment can be partially or fully observed by single agents. For multi-agent environments, see PettingZoo.

The main API methods that users of this class need to know are:

- `step()` - Updates an environment with actions returning the next agent observation, the reward for taking that actions, if the environment has terminated or truncated due to the latest action and information from the environment about the step, i.e. metrics, debug info.
- `reset()` - Resets the environment to an initial state, required before calling step. Returns the first agent observation for an episode and information, i.e. metrics, debug info.
- `render()` - Renders the environments to help visualise what the agent see, examples modes are "human", "rgb_array", "ansi" for text.
- `close()` - Closes the environment, important when external software is used, i.e. pygame for rendering, databases

Environments have additional attributes for users to understand the implementation

- `action_space` - The Space object corresponding to valid actions, all valid actions should be contained within the space.
- `observation_space` - The Space object corresponding to valid observations, all valid observations should be contained within the space.
- `spec` - An environment spec that contains the information used to initialize the environment from `gymnasium.make()`
- `metadata` - The metadata of the environment, e.g. *{"render_modes": ["rgb_array", "human"], "render_fps": 30}*. For Jax or Torch, this can be indicated to users with *"jax"=True* or *"torch"=True*.
- `np_random` - The random number generator for the environment. This is automatically assigned during `super().reset(seed=seed)` and when assessing `np_random`.

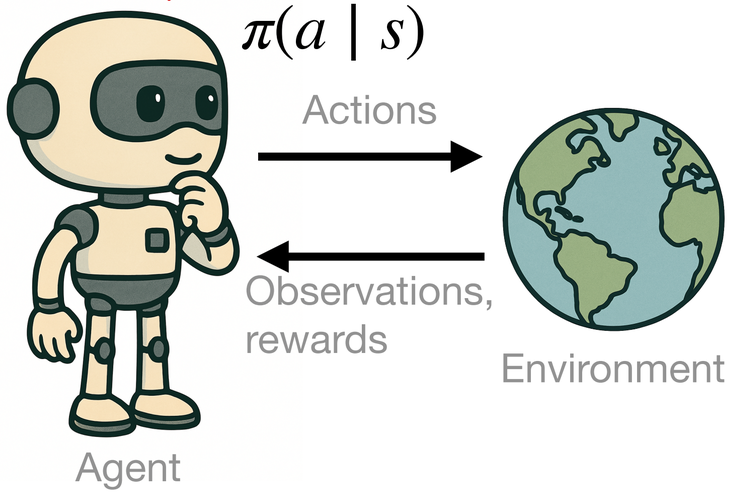
Your First RL Program

Let's start with a simple example using CartPole - perfect for understanding the basics:

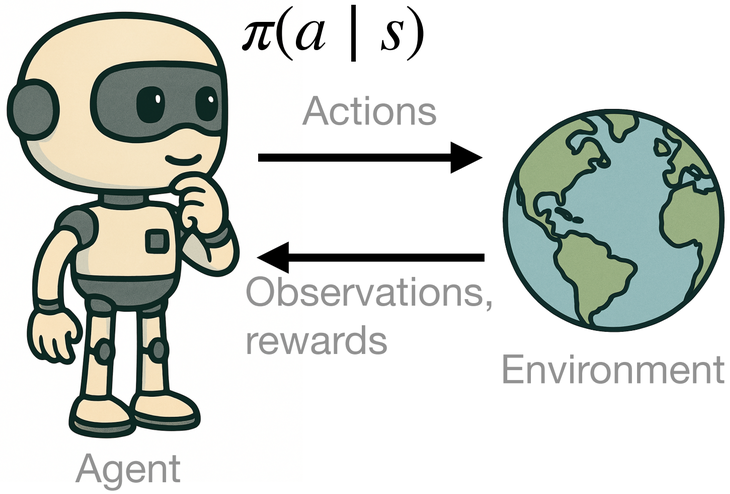
```python
# Run `pip install "gymnasium[classic-control]"` for this example.
import gymnasium as gym

# Create our training environment - a cart with a pole that needs balancing
env = gym.make("CartPole-v1", render_mode="human")

# Reset environment to start a new episode
observation, info = env.reset()
# observation: what the agent can "see" - cart position, velocity, pole angle, etc.
# info: extra debugging information (usually not needed for basic learning)

print(f"Starting observation: {observation}")
# Example output: [ 0.01234567 -0.00987654  0.02345678  0.01456789]
# [cart_position, cart_velocity, pole_angle, pole_angular_velocity]

episode_over = False
total_reward = 0

while not episode_over:
    # Choose an action: 0 = push cart left, 1 = push cart right
    action = env.action_space.sample()  # Random action for now - real agents will be smarter!

    # Take the action and see what happens
    observation, reward, terminated, truncated, info = env.step(action)

    # reward: +1 for each step the pole stays upright
    # terminated: True if pole falls too far (agent failed)
    # truncated: True if we hit the time limit (500 steps)

    total_reward += reward
    episode_over = terminated or truncated

print(f"Episode finished! Total reward: {total_reward}")
env.close()
```

Today: Monte Carlo methods for control

fix policy π



Today: Monte Carlo methods for control



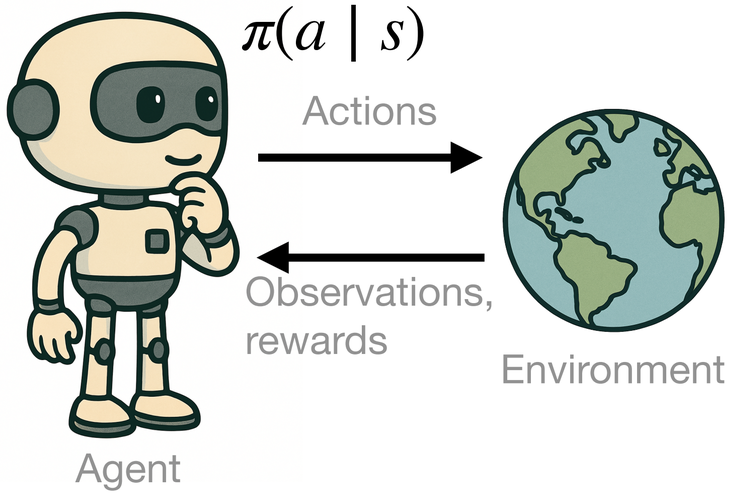
$$\Longrightarrow$$

$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

$$\vdots$$

Today: Monte Carlo methods for control



$$\Longrightarrow$$

$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

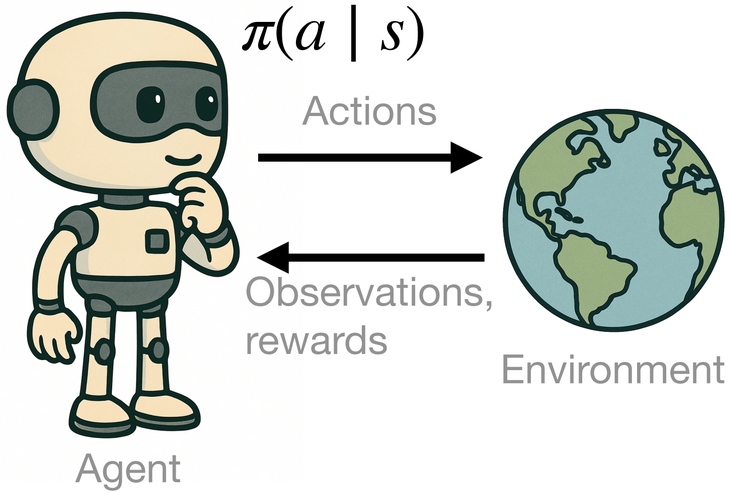
$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

$$\vdots$$

$$\Downarrow$$

Monte Carlo estimation of $\hat{V}^{\pi}(s)$

Today: Monte Carlo methods for control



$$\Longrightarrow$$

$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

$$\vdots$$

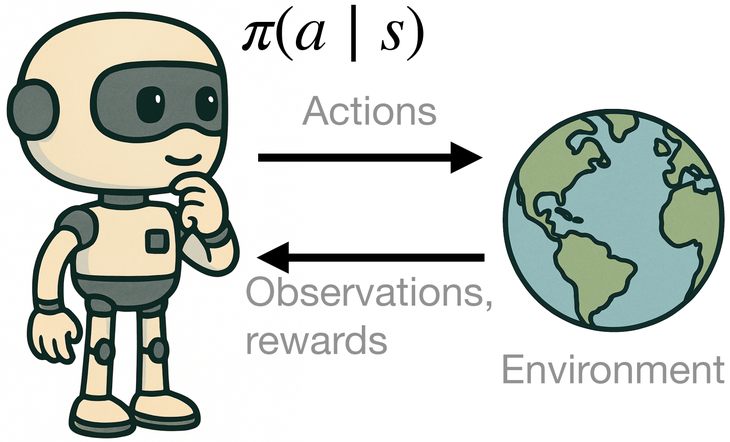
$$\Downarrow$$

Monte Carlo estimation of $\hat{V}^\pi(s)$

$$\Longleftarrow$$

Use estimate of $V^\pi(s)$ to improve $\pi \rightarrow \pi'$

Today: Monte Carlo methods for control



$$\Longrightarrow$$

$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

$$\vdots$$

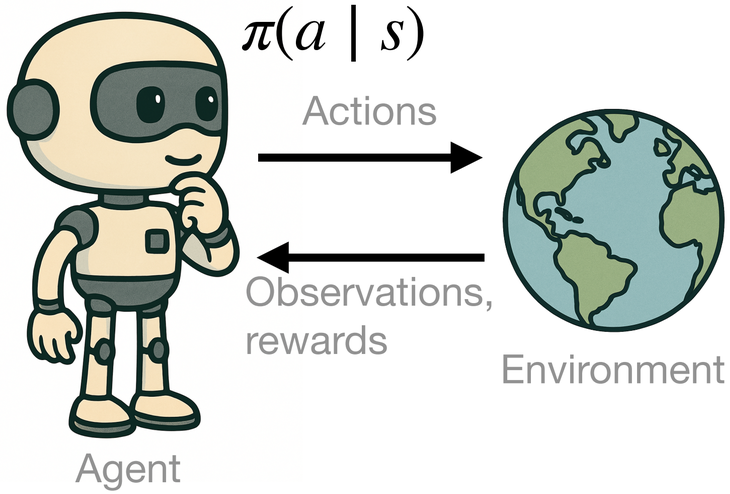
$$\Uparrow \qquad \Downarrow$$

Use estimate of $V^\pi(s)$ to improve $\pi \rightarrow \pi'$

$$\Longleftarrow$$

Monte Carlo estimation of $\hat{V}^\pi(s)$

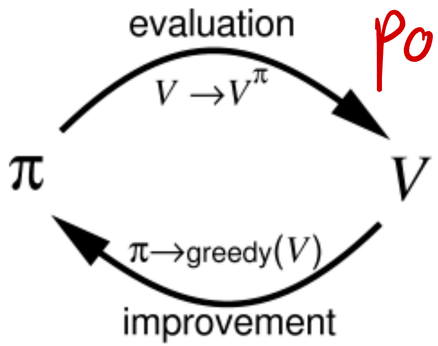
Today: Monte Carlo methods for control



$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

$\vdots$



policy iteration!

Figure courtesy of Sutton & Barto (2018)

Use estimate of $V^\pi(s)$ to improve $\pi \to \pi'$

Monte Carlo estimation of $\hat{V}^\pi(s)$

Generalized Policy Iteration

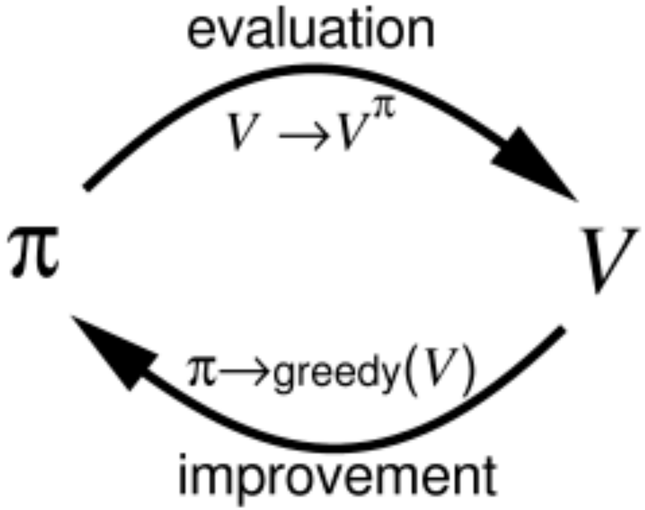


Figure courtesy of Sutton & Barto (2018)

Policy iteration: (in dynamic programming)

- Start with some policy π.
- Policy evaluation: compute $V^\pi(s)$.
- Policy improvement: make π' greedy w.r.t. $V^\pi(s)$.
- Repeat until $\pi \to \pi^*$

Generalized Policy Iteration

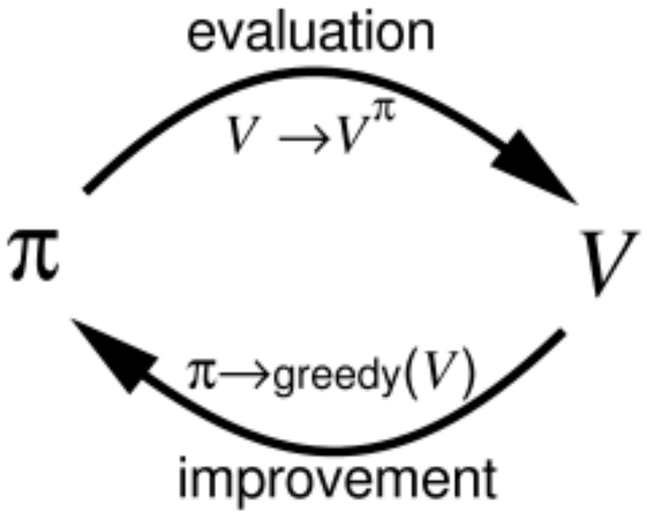


Figure courtesy of Sutton & Barto (2018)

Policy iteration:

- Start with some policy π.
- Policy evaluation: compute $V^\pi(s)$.
- Policy improvement: make π' greedy w.r.t. $V^\pi(s)$.
- Repeat until $\pi \to \pi^*$

Generalized policy iteration:

- Policy evaluation and policy iteration steps don't necessarily need to run to completion.
- These processes can happen simultaneously and approximately.
- The two processes interact, and drive the policy towards optimality.

Generalized Policy Iteration

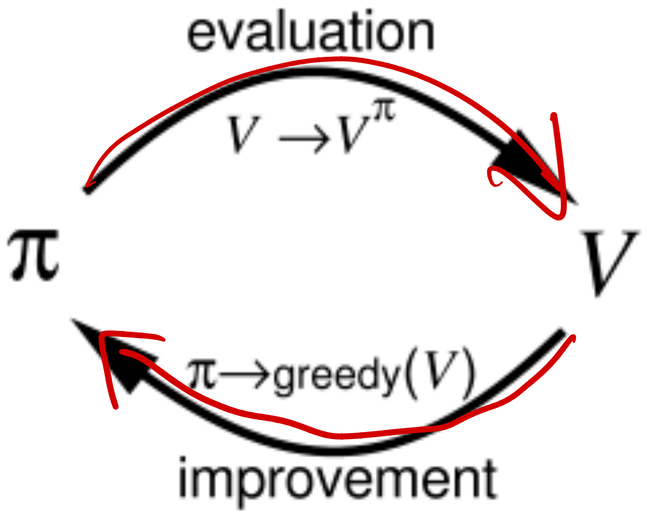


Figure courtesy of Sutton & Barto (2018)

Policy iteration:

- Start with some policy π.
- Policy evaluation: compute $V^\pi(s)$.
- Policy improvement: make π' greedy w.r.t. $V^\pi(s)$.
- Repeat until $\pi \to \pi^*$

Generalized policy iteration:

- Policy evaluation and policy iteration steps don't necessarily need to run to completion.
- These processes can happen simultaneously and approximately.
- The two processes interact, and drive the policy towards optimality.

Most RL algorithms can be viewed as different implementations of this one conceptual framework!

e.g., We earlier described value iteration as one single iteration of policy evaluation, followed by an immediate greedy policy improvement step.

Today: use environment interactions to approximate policy evaluation, then do an ϵ-greedy policy update (improve the policy a little given the value function estimate).

Today: Monte Carlo methods for control

$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

$$\vdots$$

Monte Carlo estimation of $\hat{V}^{\pi}(s)$

Use estimate of $V^{\pi}(s)$ to improve $\pi \to \pi'$

Monte Carlo methods (in general)

Suppose we want to estimate

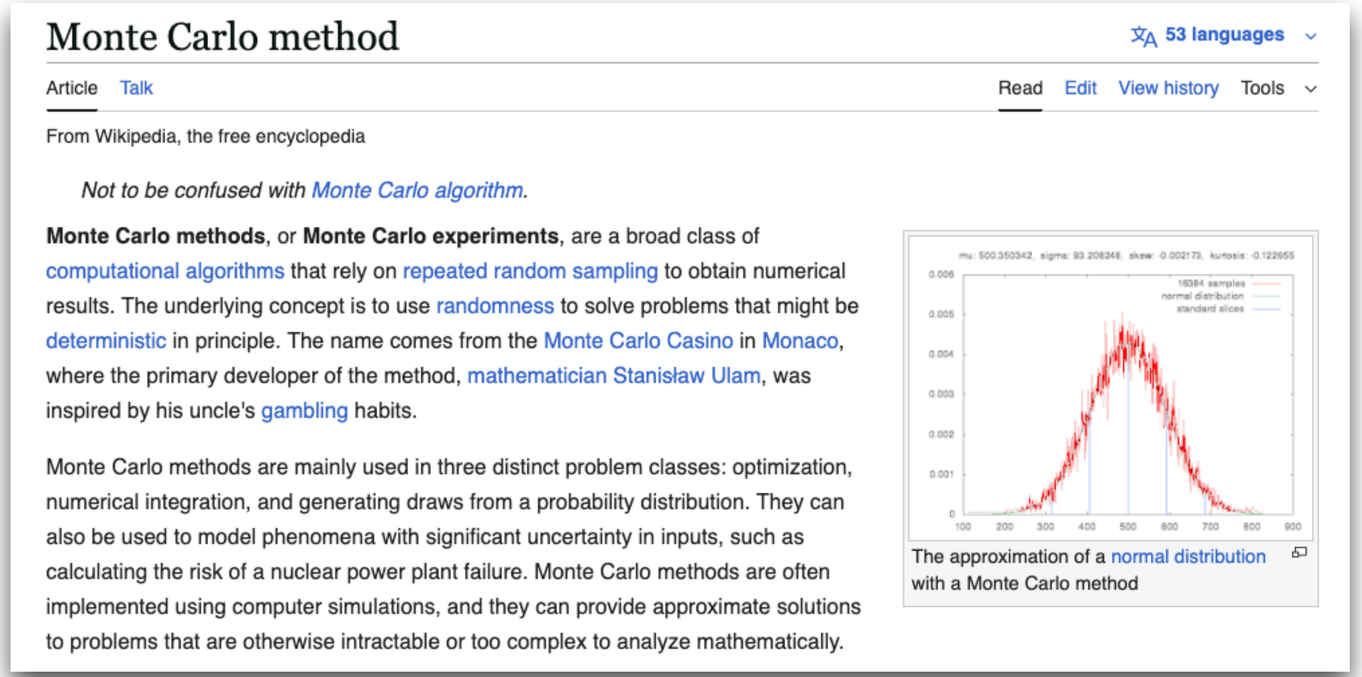
$$\mu = \mathbb{E}[X]$$

For some random variable X with a potentially unknown distribution.

Monte Carlo method: draw many i.i.d. samples $X^{(1)}, X^{(2)}, \ldots X^{(n)}$ of X, and compute the average

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^{n} X^i$$

By the law of large numbers, $\hat{\mu}_n \to \mu$ as $n \to \infty$.



Monte Carlo method

53 languages

Article Talk | Read Edit View history Tools

From Wikipedia, the free encyclopedia

Not to be confused with Monte Carlo algorithm.

Monte Carlo methods, or **Monte Carlo experiments**, are a broad class of computational algorithms that rely on repeated random sampling to obtain numerical results. The underlying concept is to use randomness to solve problems that might be deterministic in principle. The name comes from the Monte Carlo Casino in Monaco, where the primary developer of the method, mathematician Stanisław Ulam, was inspired by his uncle's gambling habits.

Monte Carlo methods are mainly used in three distinct problem classes: optimization, numerical integration, and generating draws from a probability distribution. They can also be used to model phenomena with significant uncertainty in inputs, such as calculating the risk of a nuclear power plant failure. Monte Carlo methods are often implemented using computer simulations, and they can provide approximate solutions to problems that are otherwise intractable or too complex to analyze mathematically.

The approximation of a normal distribution with a Monte Carlo method

Monte Carlo value estimation

Given π, we want: $V^{\pi}(s) = \mathbb{E}[G_t \,|\, S_t = s]$, where $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$

Monte Carlo value estimation

Given π, we want: $V^{\pi}(s) = \mathbb{E}[G_t \,|\, S_t = s]$, where $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$

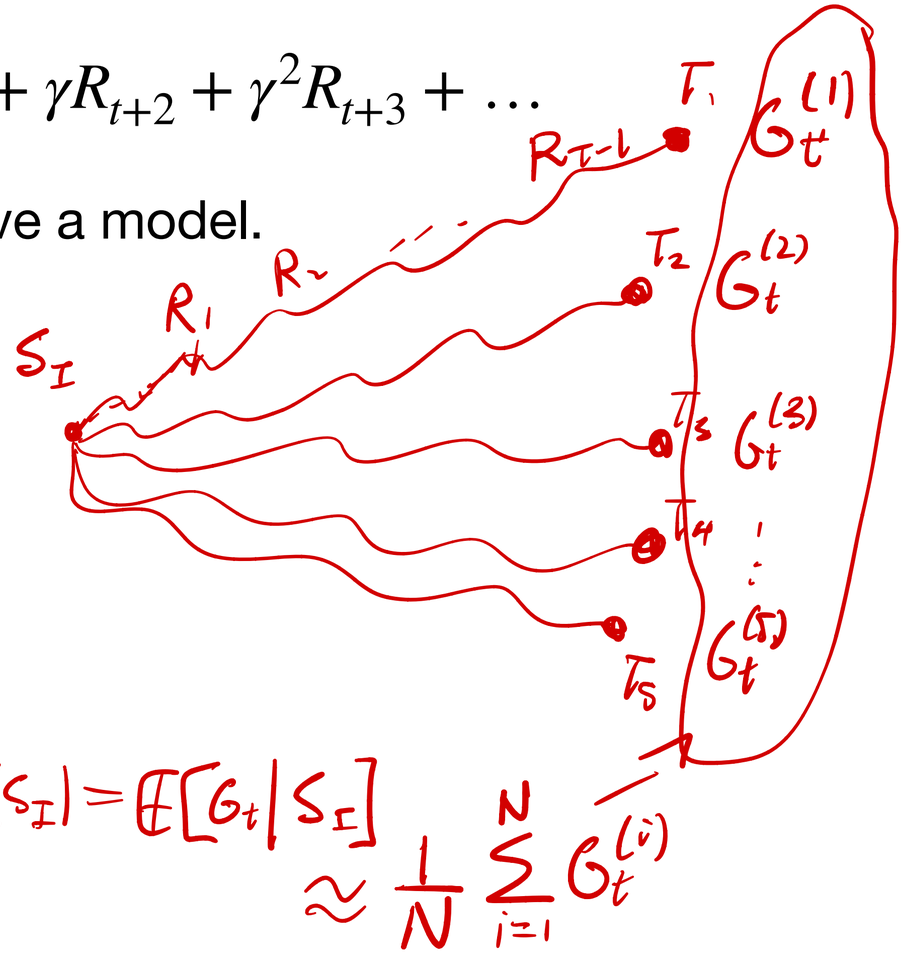
Setting: We can't compute this exactly because we don't have a model.

Monte Carlo value estimation

Given π, we want: $V^{\pi}(s) = \mathbb{E}[G_t \,|\, S_t = s]$, where $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$

Setting: We can't compute this exactly because we don't have a model.

Solution: simulate trajectories of policy π in the environment, and use the Monte Carlo approach to estimate $\hat{V}^{\pi}(s)$.



Monte Carlo value estimation

Given π, we want: $V^\pi(s) = \mathbb{E}[G_t \,|\, S_t = s]$, where $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$

Setting: We can't compute this exactly because we don't have a model.

Solution: simulate trajectories of policy π in the environment,
and use the Monte Carlo approach to estimate $\hat{V}^\pi(s)$.

E.g., if we wanted to estimate $\hat{V}^\pi(s_I)$ from one particular initial state $s_I \in S$:

1. Sample N episodes $\tau^{(i)} = s_0^{(i)} a_0^{(i)} r_0^{(i)} s_1^{(i)} a_1^{(i)} r_1^{(i)} \ldots s_{T^{(i)}}^{(i)}$ under policy π.
2. Compute realizations $g^{(i)} = r_0^{(i)} + \gamma r_1^{(i)} + \gamma^2 r_2^i + \ldots + \gamma^{T^{(i)}-1} r_{T^{(i)}-1}^{(i)}$ of G.
3. Estimate the value function at initial state $s_0 = s$ as

$$\hat{V}^\pi(s_I) = \frac{1}{N} \sum_{i=1}^{N} g^i$$

$T^{(i)}$ is the length of sampled episode i.

How are we estimating infinite sums from finite episodes?

Episodic environments: Episode is a trajectory ending in some terminal state (or after a fixed horizon).

↳ If terminal state reached, rewards are zero thereafter.

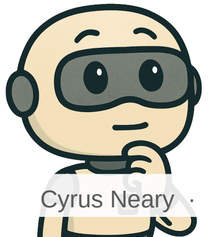
As long as episodes eventually end in terminal state, estimating values from finite trajectories/samples is no problem.

Continuing tasks: where there are no terminal states.

Having a finite episode means we truncated the episode at some point.

↳ estimating G_t with small $t \ll T$ and $\gamma < 1$, means later contributions to cumulative reward will be small / not matter.

↳ on the other hand, imagine $\gamma = 1$ or γ close to 1 and estimating G_t with t close to T.



Monte Carlo prediction

Instead of using simulated episodes to compute $\hat{V}^{\pi}(s_I)$ from a single initial state s_I, can we use them to estimate $\hat{V}^{\pi}(s)$ for every state s seen along the trajectory?

Monte Carlo prediction

Instead of using simulated episodes to compute $\hat{V}^{\pi}(s_I)$ from a single initial state s_I, can we use them to estimate $\hat{V}^{\pi}(s)$ for every state s seen along the trajectory?

Idea: Use π to sample episodes $\tau^{(i)} = s_0^{(i)} a_0^{(i)} r_0^{(i)} s_1^{(i)} a_1^{(i)} r_1^{(i)} \ldots s_{T^{(i)}}^{(i)}$.

For every $t \in \{0,1,2,\ldots T-1\}$, compute $g_t^{(i)} = r_{t+1}^{(i)} + \gamma r_{t+2}^{(i)} + \ldots + \gamma^{T^{(i)}-t-1} r_{T^{(i)}-1}$.

For each $s \in S$, estimate $\hat{V}^{\pi}(s) = \frac{1}{N(s)} \sum_{i=1}^{N} \sum_{t=0}^{T} \mathbb{I}\{S_t = s\} g_t^{(i)}$

Monte Carlo prediction

Instead of using simulated episodes to compute $\hat{V}^{\pi}(s_I)$ from a single initial state s_I, can we use them to estimate $\hat{V}^{\pi}(s)$ for every state s seen along the trajectory?

Idea: Use π to sample episodes $\tau^{(i)} = s_0^{(i)} a_0^{(i)} r_0^{(i)} s_1^{(i)} a_1^{(i)} r_1^{(i)} \ldots s_{T^{(i)}}^{(i)}$.
For every $t \in \{0,1,2,\ldots T-1\}$, compute $g_t^{(i)} = r_{t+1}^{(i)} + \gamma r_{t+2}^{(i)} + \ldots + \gamma^{T^{(i)}-t-1} r_{T^{(i)}-1}$.
For each $s \in S$, estimate $\hat{V}^{\pi}(s) = \frac{1}{N(s)} \sum_{i=1}^{N} \sum_{t=0}^{T} \mathbb{I}\{S_t = s\} g_t^{(i)}$

First-visit MC: In each episode, if we hit state s multiple times, we only count the return from the first time we see it.

Every-visit MC: Count the return after every time we see s in an episode.

Monte Carlo prediction

Instead of using simulated episodes to compute $\hat{V}^\pi(s_I)$ from a single initial state s_I, can we use them to estimate $\hat{V}^\pi(s)$ for every state s seen along the trajectory?

Idea: Use π to sample episodes $\tau^{(i)} = s_0^{(i)} a_0^{(i)} r_0^{(i)} s_1^{(i)} a_1^{(i)} r_1^{(i)} \ldots s_{T^{(i)}}^{(i)}$.
For every $t \in \{0,1,2,\ldots T-1\}$, compute $g_t^{(i)} = r_{t+1}^{(i)} + \gamma r_{t+2}^{(i)} + \ldots + \gamma^{T^{(i)}-t-1} r_{T^{(i)}-1}$.
For each $s \in S$, estimate $\hat{V}^\pi(s) = \frac{1}{N(s)} \sum_{i=1}^{N} \sum_{t=0}^{T} \mathbb{I}\{S_t = s\} g_t^{(i)}$

First-visit MC: In each episode, if we hit state s multiple times, we only count the return from the first time we see it.

Every-visit MC: Count the return after every time we see s in an episode.

Both first-visit and every-visit MC converge to $V^\pi(s)$ as the number of (first) visits to s tends to infinity.

why? Law of large numbers.

Monte Carlo prediction

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Figure courtesy of Sutton & Barto (2018).

Monte Carlo prediction

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Figure courtesy of Sutton & Barto (2018).

Every-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

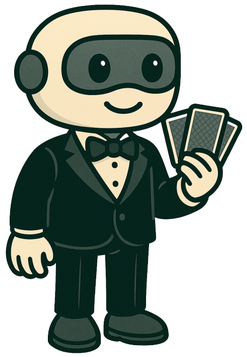
$G \leftarrow \gamma G + R_{t+1}$

~~Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:~~

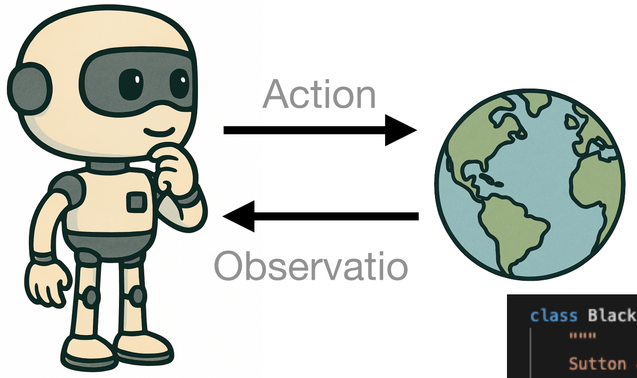
Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Example of Monte Carlo prediction: blackjack



Player policy: stick if player's sum is 20 or 21, otherwise hit.



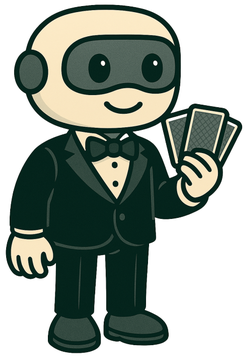
Gymnasium

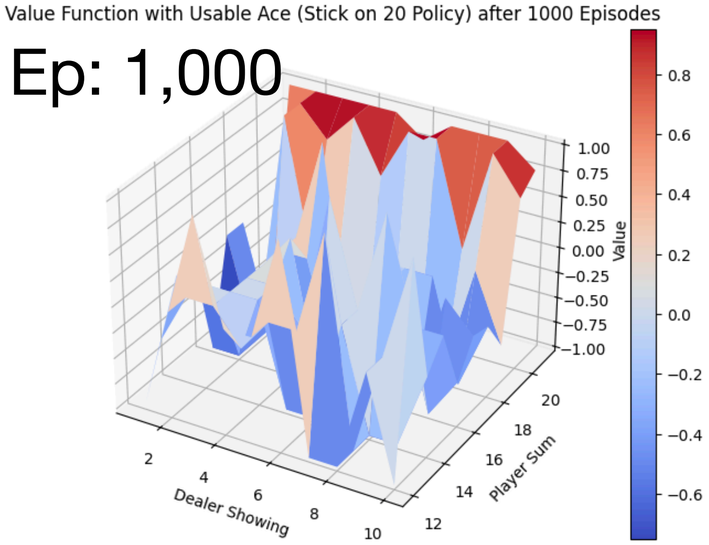
```python
for _ in range(num_episodes):
    states, actions, rewards = generate_episode(env, policy)

    # Compute returns G_t (works for general gamma)
    G = 0.0
    returns = [0.0] * len(rewards)
    for t in reversed(range(len(rewards))):
        G = rewards[t] + gamma * G
        returns[t] = G

    # First-visit updates
    seen = set()
    for t, s in enumerate(states):
        if s in seen:
            continue
        seen.add(s)
        ps_i, ds_i, ua_i = s  # already 0-based
        N[ps_i, ds_i, ua_i] += 1
        # Incremental mean update: V <- V + (G - V)/N
        V[ps_i, ds_i, ua_i] += (returns[t] - V[ps_i, ds_i, ua_i]) / N[ps_i, ds_i, ua_i]
```

```python
class BlackjackEnv(gym.Env):
    """
    Sutton & Barto blackjack, Gymnasium-compatible.

    Observation: (player_sum, dealer_upcard, usable_ace)
        - player_sum in [12..21] (we auto-hit under 12 on reset)
        - dealer_upcard in [1..10] (1 = Ace, 10 = {10,J,Q,K})
        - usable_ace in {0,1}
    Action: Discrete(2)  (0=STICK, 1=HIT)
    Reward: terminal only: +1 win / -1 loss / 0 draw. gamma=1.
    Dealer: hits <17, sticks on 17+ (including soft 17).
    Deck: infinite (with replacement). P(Ace)=1/13, P(2..9)=1/13 each, P(10)=4/13.
    """
    metadata = {"render_modes": ["human", "ansi"]}

    def __init__(self, seed: Optional[int] = None):
        super().__init__()
        # Observation as Tuple(Discrete, Discrete, Discrete), like Gym classic
        self.observation_space = spaces.Tuple((
            spaces.Discrete(10),  # player sum ∈ {12,...,21} (encoded as 0–9)
            spaces.Discrete(10),  # dealer upcard ∈ {1,...,10} (encoded as 0–9)
            spaces.Discrete(2),   # usable ace ∈ {0,1}
        ))
        self.action_space = spaces.Discrete(2)  # 0=STICK, 1=HIT
        self.np_random, _ = np_random(seed)

        self.player: List[int] = []
        self.dealer: List[int] = []
        self._pending_done: bool = False   # for immediate naturals after reset
        self._pending_reward: float = 0.0
        self._terminated: bool = False
```

Example of Monte Carlo prediction: blackjack

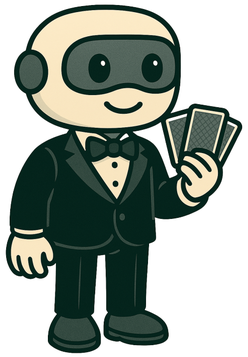
Player policy: stick if player's sum is 20 or 21, otherwise hit.



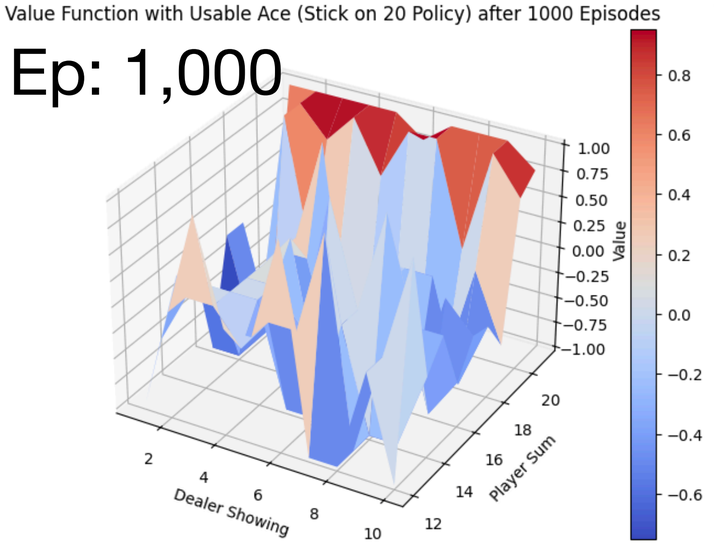


Example of Monte Carlo prediction: blackjack

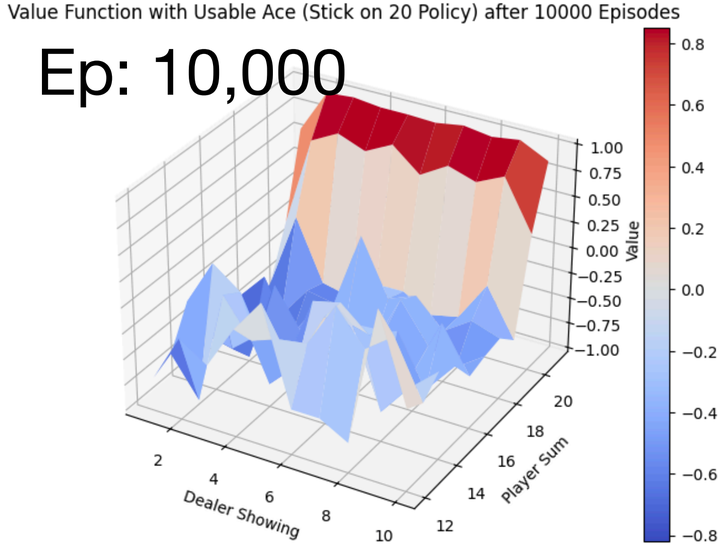
Player policy: stick if player's sum is 20 or 21, otherwise hit.



Ep: 1,000

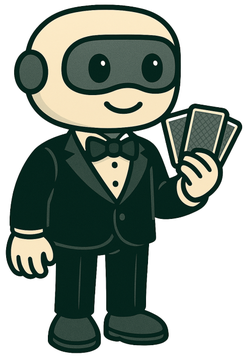


Ep: 10,000



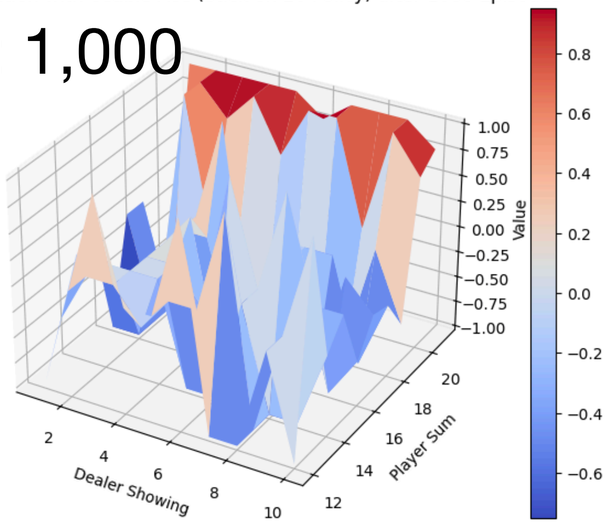
Example of Monte Carlo prediction: blackjack

Player policy: stick if player's sum is 20 or 21, otherwise hit.



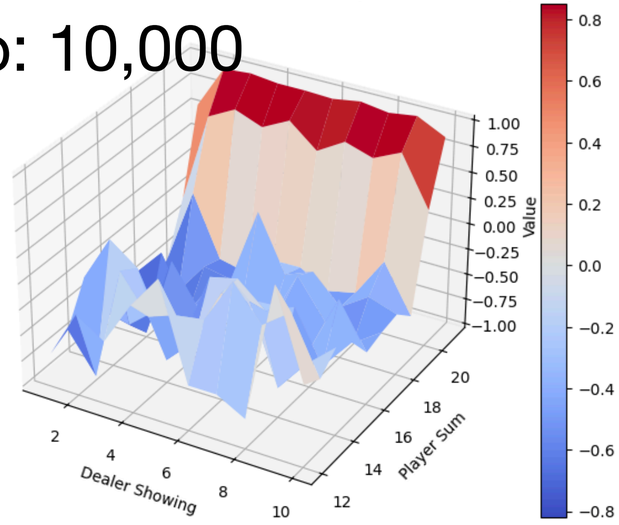
Value Function with Usable Ace (Stick on 20 Policy) after 1000 Episodes

Ep: 1,000



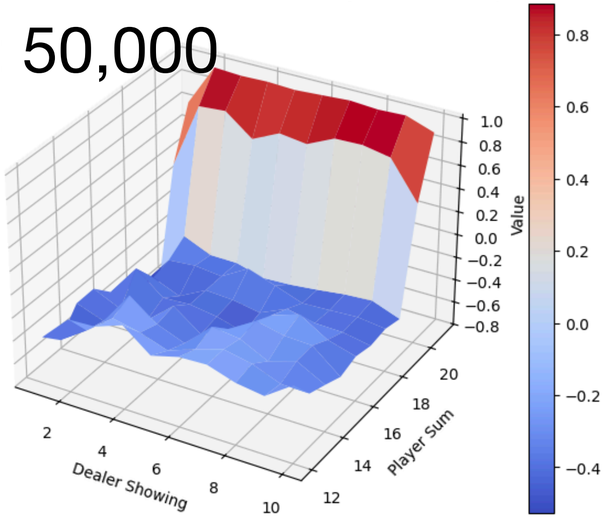
Value Function with Usable Ace (Stick on 20 Policy) after 10000 Episodes

Ep: 10,000



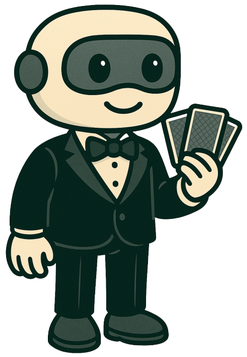
Value Function with Usable Ace (Stick on 20 Policy) after 50000 Episodes

Ep: 50,000

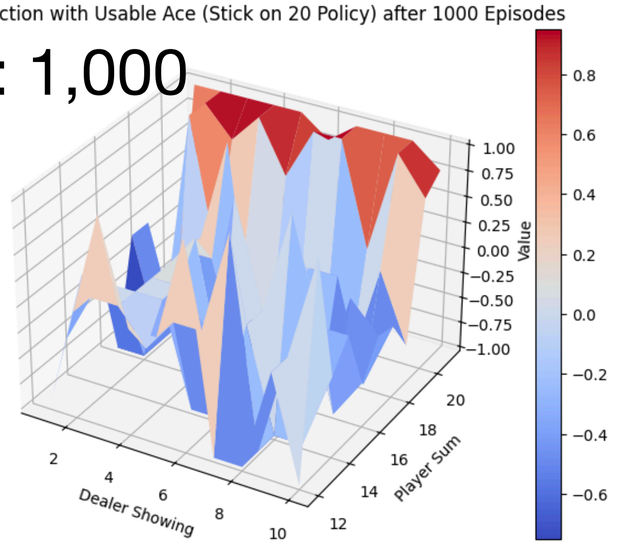


Example of Monte Carlo prediction: blackjack

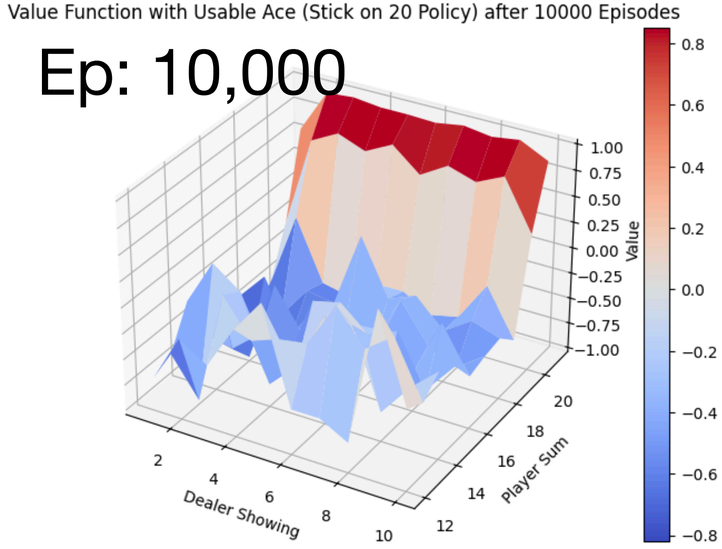
Player policy: stick if player's sum is 20 or 21, otherwise hit.



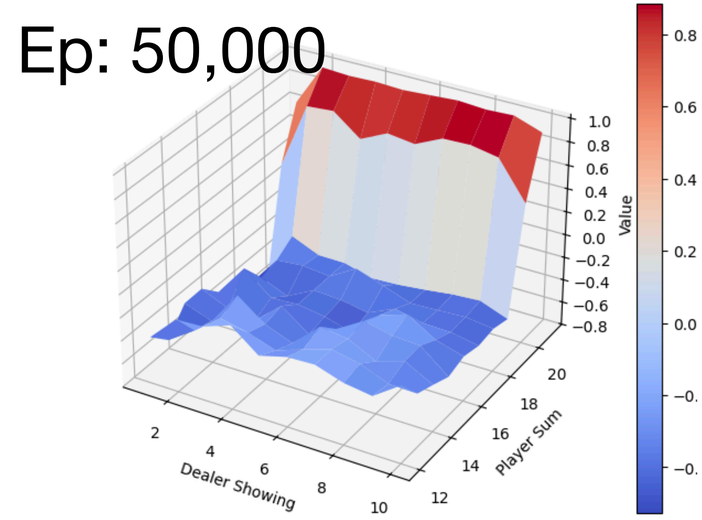
Ep: 1,000



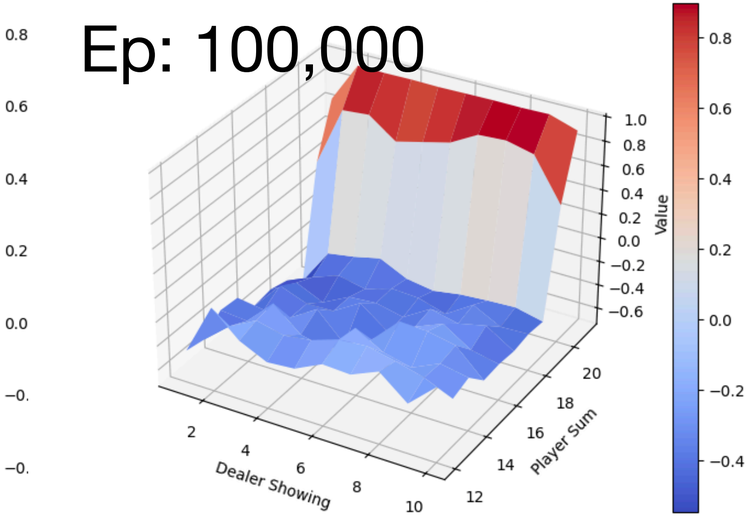
Ep: 10,000



Ep: 50,000



Ep: 100,000



do the same estimate of $\hat{V}^{\pi}(s)$

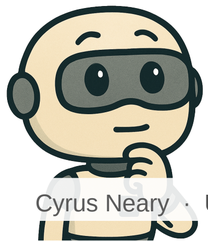
Could ~~we solve the Blackjack problem with~~ dynamic programming?

- In principle yes. But, it's a lot harder.

  Making a blackjack sim is easy.

  Writing out a transition function that is correct is harder.

takeaway: It's very useful to only need samples from simulator/real world, not an explicit transition function.



How else is Monte Carlo prediction different from dynamic programming?

DP:
- Recursively, using one-step transitions and values at next state to update every state's value systematically.
- Estimates of value at each state depend on the estimates at neighboring states through Bellman's equations.

MC:
- Each update looks all the way to the end of an episode, to update the current state value estimate.
- Seperate value estimates are somewhat independent.
- Computational expense of estimating value from given states doesn't depend on the size of the MDP.

As a result:
- DP spreads information everywhere with each sweep.
- MC spreads information only where we have been. (doesn't need model).

How can we extract a greedy policy from $\hat{V}^{\pi}(s)$ without an MDP model?

Given the MDP model, we could compute a policy improvement step:

$$\pi'(s) = \arg\max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) \hat{V}^{\pi}(s) \right]$$

What do we do when we can't evaluate $R(s,a)$ and $T(s' \mid s, a)$?

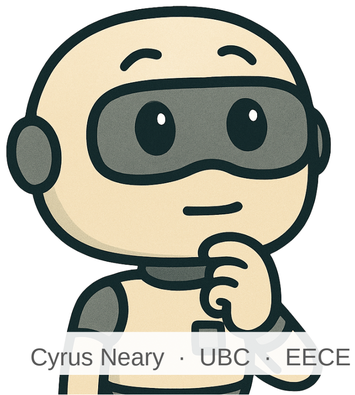
option 1
we could infer $\hat{R}$ and $\hat{T}$ from data, and evaluate π' Model-Based RL

option 2
we could estimate $\hat{Q}^{\pi}(s,a)$ the state-action values instead of $\hat{V}^{\pi}(s)$.

Then $\pi'(s) = \arg\max_{a \in A} \hat{Q}(s,a)$.

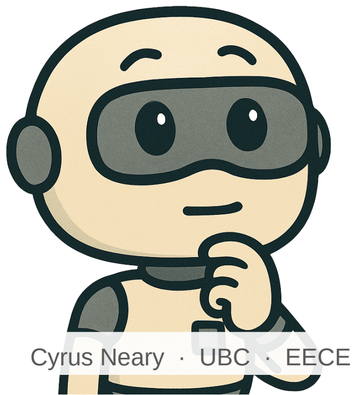
Which of the following does Monte Carlo prediction **not** require?

A: The ability to generate episodes by following π.
B: The rewards observed along each episode.
C: The transition probabilities $T(s' \mid s, a)$.
D: Episodes that eventually terminate (or are cut off at a fixed horizon).
E: I don't understand.



You have a simulator of a system with about 10^{12} states, and you only care about $V^{\pi}(s)$ at a single start state. Which of the following is true?

A: DP is preferable, since it doesn't need any sampled episodes.
B: MC can estimate it by averaging returns from episodes started at that state, and the cost doesn't depend on the total number of states.
C: MC must first estimate $V^{\pi}(s)$ at every reachable state before it can estimate the start state.
D: Neither method can work without an explicit transition function.
E: I don't understand.



In a continuing task, you truncate each episode after T steps and run MC prediction with $\gamma < 1$. Which **sampled returns** g_t are the most biased estimates of $V^\pi(s_t)$?

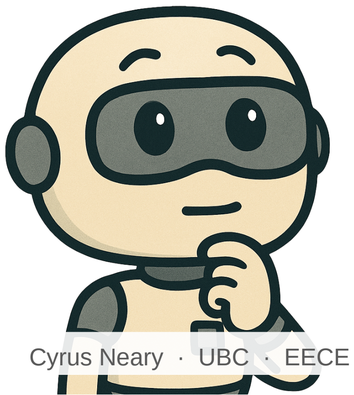
A: Returns from early time steps, because they sum the most noisy rewards.

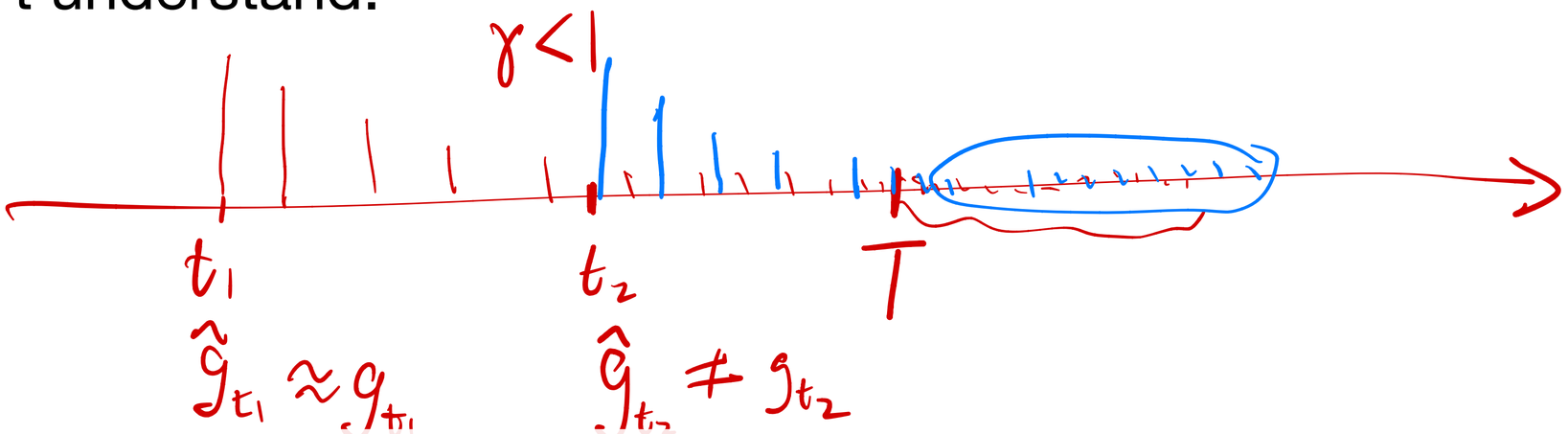
B: All returns equally, because $\gamma < 1$ discounts every return the same way.

C: None, because the law of large numbers guarantees convergence.

D: Returns from time steps close to the truncation point, because most of their future rewards are cut off.

E: I don't understand.





Monte Carlo estimation of action values

$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

$$\vdots$$

$$\Longrightarrow$$

Monte Carlo estimation of $\hat{Q}^{\pi}(s, a)$

Monte Carlo estimation of action values

$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$
$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$
$$\vdots$$

$\Longrightarrow$

Monte Carlo estimation of $\hat{Q}^\pi(s, a)$

Method: Use the same Monte Carlo technique as we used to estimate $\hat{V}^\pi(s)$. Now, consider visits to state-action pairs (s, a). We say a pair (s, a) is visited in an episode if action a was taken from state s.

Much less data per "object" when considering (s,a) vs. s.

Monte Carlo estimation of action values

$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$
$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$
$$\vdots$$

$\Longrightarrow$

Monte Carlo estimation of $\hat{Q}^\pi(s, a)$

Method: Use the same Monte Carlo technique as we used to estimate $\hat{V}^\pi(s)$. Now, consider visits to state-action pairs (s, a). We say a pair (s, a) is visited in an episode if action a was taken from state s.

Challenge: Many state-action pairs may never be visited.

- E.g., if π is a deterministic policy, only one action will ever be observed from each state. This defeats the purpose of learning a state-action value in the first place (to differentiate between values of different actions).

Monte Carlo estimation of action values

$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$
$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$
$$\vdots$$

$\Longrightarrow$

Monte Carlo estimation of $\hat{Q}^\pi(s, a)$

Method: Use the same Monte Carlo technique as we used to estimate $\hat{V}^\pi(s)$. Now, consider visits to state-action pairs (s, a). We say a pair (s, a) is visited in an episode if action a was taken from state s.

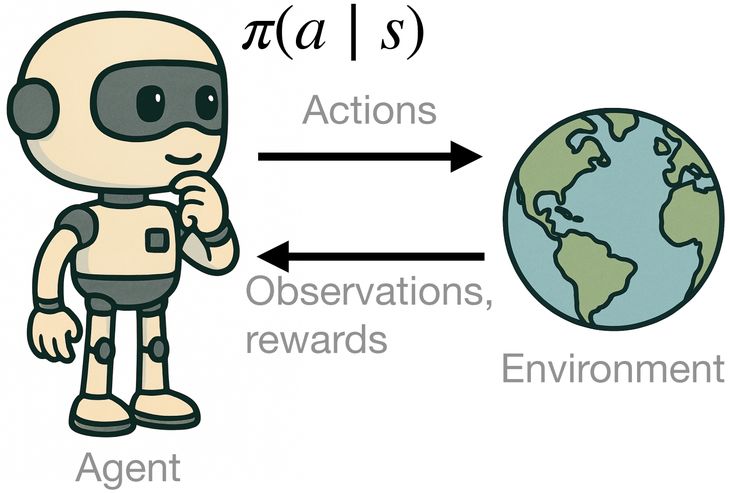
Challenge: Many state-action pairs may never be visited.

- E.g., if π is a deterministic policy, only one action will ever be observed from each state. This defeats the purpose of learning a state-action value in the first place (to differentiate between values of different actions).

This is an exploration problem.

Solution 1: (*exploring starts*) Assume each state-action pair is used to start the episode with *some* non-zero probability.

Monte Carlo control via GPI



$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

$$\vdots$$

$$\pi'(s) = \arg\max_{a \in A} \hat{Q}^{\pi}(s, a)$$

Use estimate of $\hat{Q}^{\pi}(s, a)$ to improve $\pi \rightarrow \pi'$

Monte Carlo estimation of $\hat{Q}^{\pi}(s, a)$

Monte Carlo control with the exploring starts assumption

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$

$Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Loop forever (for each episode):

Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0

Generate an episode from S_0, A_0, following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

Figure courtesy of Sutton & Barto (2018).

Core ideas:

- (GPI) Instead of sampling enough for the policy evaluation step to converge to the true $Q^{\pi_k}(s,a)$, just sample enough to move the estimate *towards* this value.
- One natural choice: alternate between policy evaluation and improvement after ever episode.

Note: returns for each state-action pair are accumulated and averaged, irrespective of what policy was used to gather that observation.

How can we reason this algorithm will ~~not~~ converge to a sub-optimal policy?

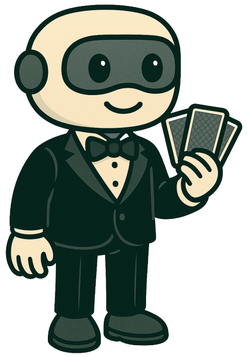
/will not

We can say:

↳ If it converges, it converges to Q^* / π^*.

But not necessarily that it will converge.

Solving Blackjack with MC control + exploring starts



Initial policy π_0: stick if player's sum is 20 or 21, otherwise hit.

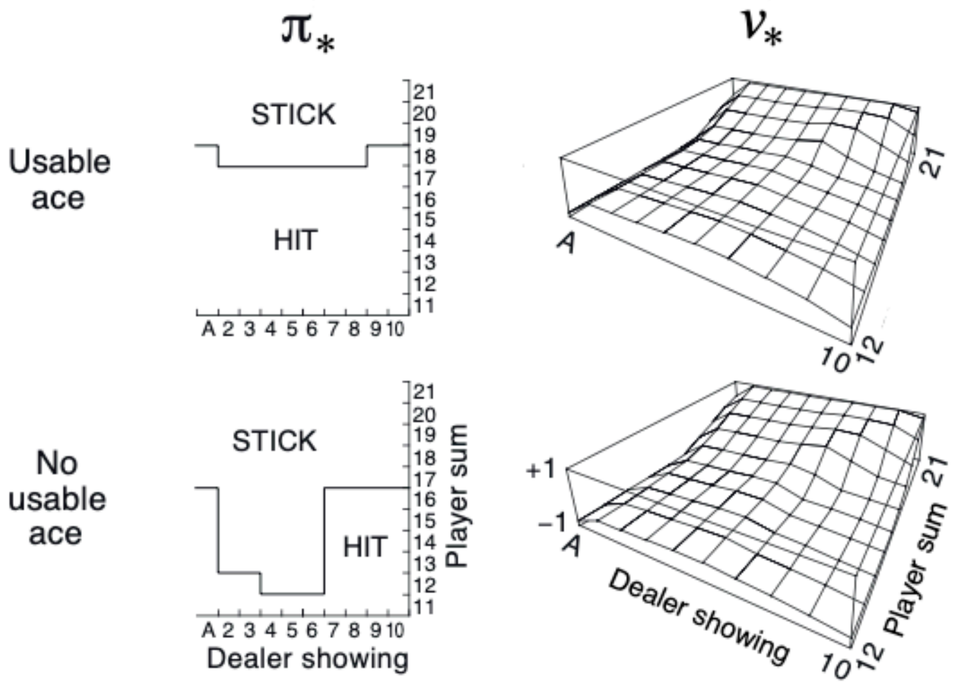


Figure 5.2: The optimal policy and state-value function for blackjack, found by Monte Carlo ES. The state-value function shown was computed from the action-value function found by Monte Carlo ES. ■

Figure courtesy of Sutton & Barto (2018). See page 100 for a full description of the Blackjack game setup.

Removing the exploring starts assumption: ϵ-soft policies

Idea: Similarly to ϵ-greedy policies in bandit algorithms, select random action with probability ϵ (ensuring exploration), and select $\arg\max_{a \in A} \hat{Q}^{\pi}(s, a)$ with probability $1 - \epsilon$.

On-policy first-visit MC control (for ε-soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ε-soft policy

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

Figure courtesy of Sutton & Barto (2018).

Removing the exploring starts assumption: ϵ-soft policies

Idea: Similarly to ϵ-greedy policies in bandit algorithms, select random action with probability ϵ (ensuring exploration), and select $\arg\max_{a \in A} \hat{Q}^{\pi}(s, a)$ with probability $1 - \epsilon$.

Introducing stochasticity guarantees exploration. For large numbers of sampled episodes, all actions will be selected from each state infinitely often, so long as each state is reachable in the MDP.

On-policy first-visit MC control (for ε-soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ε-soft policy

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

Figure courtesy of Sutton & Barto (2018).

Does the policy improvement theorem still hold?

Recall: π, π' are two policies s.t. for all $s \in \mathcal{S}$, $Q^\pi(s, \pi'(s)) \geq V^\pi(s)$, Then $V^{\pi'}(s) \geq V^\pi(s)$ for all states $s \in \mathcal{S}$.

Suppose π is some ε-soft policy with Q-function $Q^\pi(s,a)$. Let π' be an ε-greedy improvement using Q^π. Then

$$Q^\pi(s, \pi'(s)) = \sum_a \pi'(a|s)\, Q^\pi(s,a)$$

$$= \frac{\varepsilon}{|A|}\left\{\sum_a Q^\pi(s,a)\right\} + (1-\varepsilon) \operatorname*{argmax}_{a \in A} Q^\pi(s,a)$$

This must be ≥ 0

$$\geq \frac{\varepsilon}{|A|}\sum_a Q^\pi(s,a) + (1-\varepsilon)\sum_a \left(\pi(a|s) - \frac{\varepsilon}{|A|}\right) Q^\pi(s,a)$$

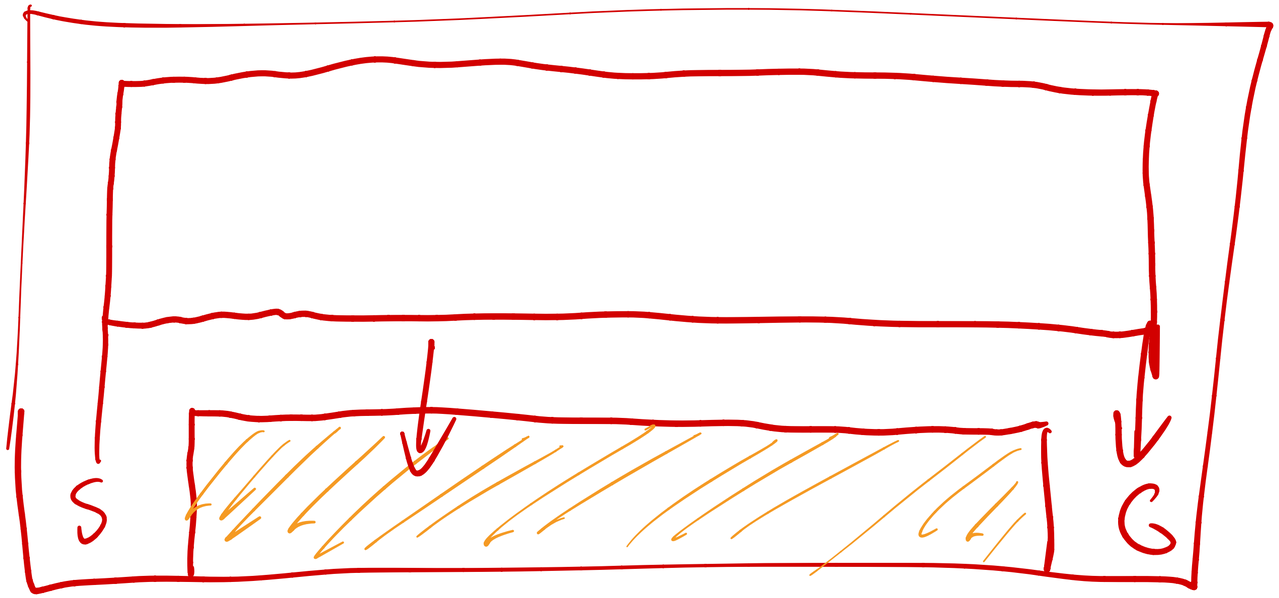
$1-\varepsilon$

$$= \frac{\varepsilon}{|A|}\sum_a Q^\pi(s,a) - \frac{\varepsilon}{|A|}\sum_a Q^\pi(s,a) + \sum_a \pi(a|s)\, Q^\pi(s,a)$$

$$= V^\pi(s)$$

$$\Rightarrow Q^\pi(s, \pi'(s)) \geq V^\pi(s)$$ ☺

Are we really learning the value of an optimal policy?



want to get to goal as quickly as possible.

Are we really learning the value of an optimal policy?

No! we are learning the $Q(s,a)$ value for some near-optimal policy.

↳ Our policy improvement won't converge to the optimal, but instead to some near-optimal ε-soft policy.

This is a compromise. We're trying to learn action values conditioned on subsequent optimal behavior, but also need to introduce non-optimal exploration to get enough data.

This is because we're using an **on-policy** algorithm.

How can we solve this?

↳ adapt exploration over time.

↳ Use data from a different policy to update current policy.

↳ off-policy learning!

On-policy vs. off-policy algorithms

On-policy learning: Learn about the policy we are actually using to generate episodes.

Behavior policy $\pi^b = \pi^t$ target policy

Intuition: Evaluate and improve what **you did** in the environment.

- simple + easy to implement.

Off-policy learning: Learn about a target policy, while following a different behavior policy.

Behavior policy $\pi^b \neq \pi^t$ target policy

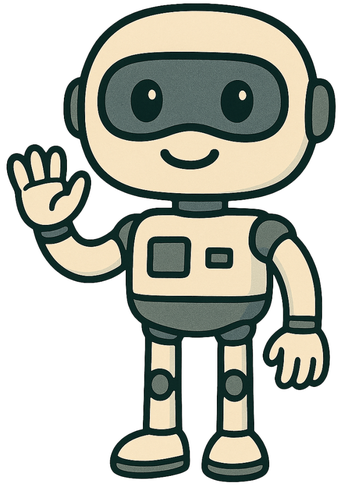
Intuition: Evaluate and improve what **you wish you did** in the environment, using data from something else.

- Requires careful consideration of the distribution shift issues.
- More general class of algorithms
- we sidestep the issue of on-policy from the last slide.

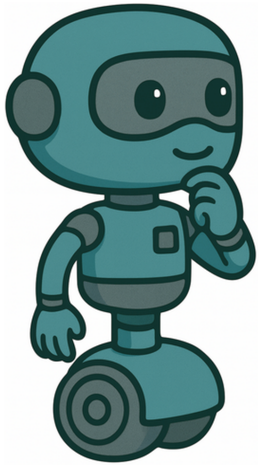
Off-policy prediction via importance sampling (SI)

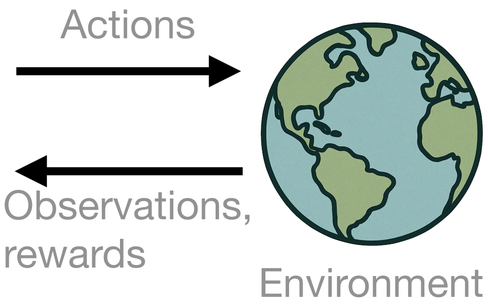
Objective: Estimate the value $\hat{v}^{\pi}(s, a)$ of *target policy* $\pi(a \mid s)$, given data generated under a different *behavior policy* $b(a \mid s)$.

Learn a target policy $\pi(a \mid s)$



Using episodes sampled under a different behavioral policy $b(a \mid s)$

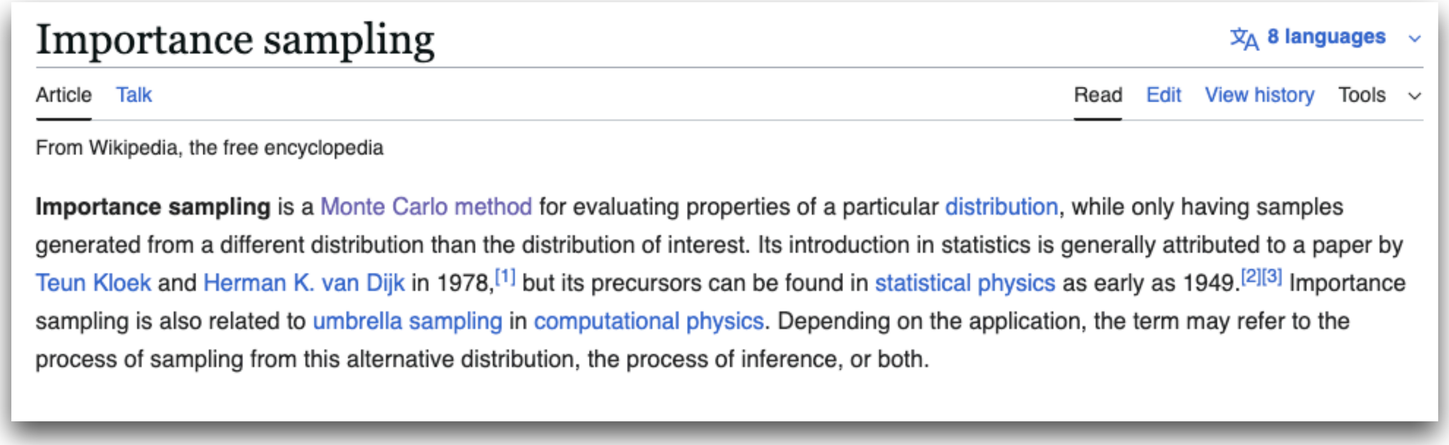




Importance sampling

Suppose we want to compute $\mathbb{E}[f(X)]$ for some r.v. X with a distribution $p(x)$ that is difficult or impossible to sample.

$$\mathbb{E}_{x \sim p}\left[f(x)\right]$$



Importance sampling

8 languages

Article Talk Read Edit View history Tools

From Wikipedia, the free encyclopedia

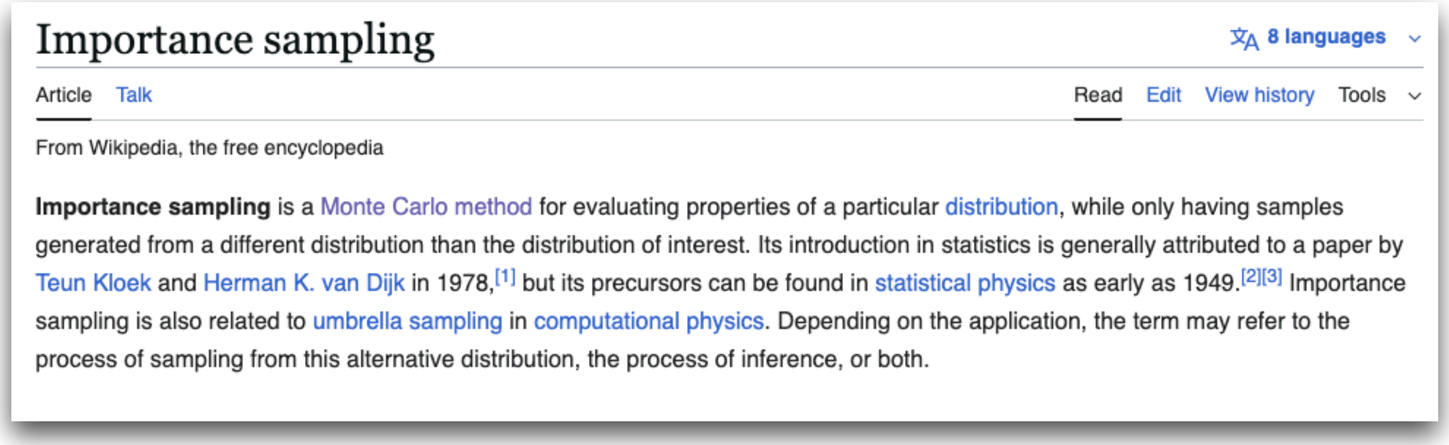
Importance sampling is a Monte Carlo method for evaluating properties of a particular distribution, while only having samples generated from a different distribution than the distribution of interest. Its introduction in statistics is generally attributed to a paper by Teun Kloek and Herman K. van Dijk in 1978,[1] but its precursors can be found in statistical physics as early as 1949.[2][3] Importance sampling is also related to umbrella sampling in computational physics. Depending on the application, the term may refer to the process of sampling from this alternative distribution, the process of inference, or both.

Instead, we are able to draw samples from another distribution $q(x)$ that is much easier to sample, or that assigns higher probability to events that are very rare under $q(\cdot)$.

Importance sampling

Suppose we want to compute $\mathbb{E}[f(X)]$ for some r.v. X with a distribution $p(x)$ that is difficult or impossible to sample.

$$\mathbb{E}_{x\sim p}\left[f(x)p(x)\right]$$



Importance sampling

8 languages

Article Talk Read Edit View history Tools

From Wikipedia, the free encyclopedia

Importance sampling is a Monte Carlo method for evaluating properties of a particular distribution, while only having samples generated from a different distribution than the distribution of interest. Its introduction in statistics is generally attributed to a paper by Teun Kloek and Herman K. van Dijk in 1978,[1] but its precursors can be found in statistical physics as early as 1949.[2][3] Importance sampling is also related to umbrella sampling in computational physics. Depending on the application, the term may refer to the process of sampling from this alternative distribution, the process of inference, or both.

Instead, we are able to draw samples from another distribution $q(x)$ that is much easier to sample, or that assigns higher probability to events that are very rare under $q(\cdot)$.

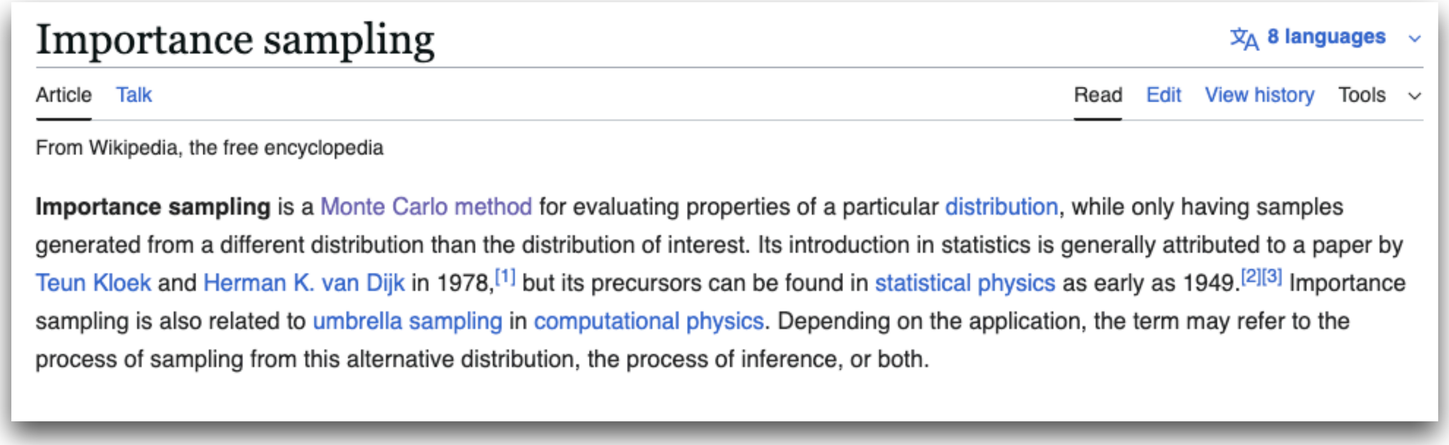
Importance sampling lets us compute $\mathbb{E}_{x\sim p}[f(X)]$ by re-weighting samples of according to the likelihood ratio $\frac{p(x)}{q(x)}$.

$$\mathbb{E}_{x\sim p}[f(X)] = \sum_{x\in\mathcal{X}} f(x)p(x) = \sum_{x\in\mathcal{X}} f(x)\frac{p(x)}{q(x)}q(x) \approx \frac{1}{N}\sum_{i=1}^{N} f(x^{(i)})\frac{p(x^{(i)})}{q(x^{(i)})}q(x^{(i)})$$

Importance sampling

Suppose we want to compute $\mathbb{E}[f(X)]$ for some r.v. X with a distribution $p(x)$ that is difficult or impossible to sample.

$$\mathbb{E}_{x\sim p}\left[f(x)p(x)\right]$$



Importance sampling

8 languages

Article Talk Read Edit View history Tools

From Wikipedia, the free encyclopedia

Importance sampling is a Monte Carlo method for evaluating properties of a particular distribution, while only having samples generated from a different distribution than the distribution of interest. Its introduction in statistics is generally attributed to a paper by Teun Kloek and Herman K. van Dijk in 1978,[1] but its precursors can be found in statistical physics as early as 1949.[2][3] Importance sampling is also related to umbrella sampling in computational physics. Depending on the application, the term may refer to the process of sampling from this alternative distribution, the process of inference, or both.

Instead, we are able to draw samples from another distribution $q(x)$ that is much easier to sample, or that assigns higher probability to events that are very rare under $q(\cdot)$.

Importance sampling lets us compute $\mathbb{E}_{x\sim p}[f(X)]$ by re-weighting samples of according to the likelihood ratio $\frac{p(x)}{q(x)}$.

$$\mathbb{E}_{x\sim p}[f(X)] = \sum_{x\in\mathcal{X}} f(x)p(x) = \sum_{x\in\mathcal{X}} f(x)\frac{p(x)}{q(x)}q(x) \approx \frac{1}{N}\sum_{i=1}^{N} f(x^{(i)})\frac{p(x^{(i)})}{q(x^{(i)})}q(x^{(i)})$$

Note: we must assume overlapping support, $\forall x,\ p(x) > 0 \implies q(x) > 0$.
Also, $q(x)$ should be similar to $p(x)$, otherwise the estimator will have very high variance, and will not be very useful.

Importance sampling for off-policy Monte Carlo evaluation

Recall that our goal is to estimate $V^{\pi}(s) = \mathbb{E}\left[G_t \mid S_t = s, \pi\right] = \sum_{\tau} G_t Pr(a_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, \pi)$,

but we would like to use episodes $\tau^{(i)} = s_0^{(i)}, a_0^{(i)}, r_0^{(i)}, s_1^{(i)} \ldots s_{T^{(i)}}^{(i)}$ sampled under a different behavioral policy $b(a \mid s)$.

Importance sampling for off-policy Monte Carlo evaluation

Recall that our goal is to estimate $V^{\pi}(s) = \mathbb{E}\left[G_t \mid S_t = s, \pi\right] = \sum_{\tau} G_t Pr(a_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, \pi)$,

but we would like to use episodes $\tau^{(i)} = s_0^{(i)}, a_0^{(i)}, r_0^{(i)}, s_1^{(i)} \ldots s_{T^{(i)}}^{(i)}$ sampled under a different behavioral policy $b(a \mid s)$.

Consider the probability $Pr(a_t, r_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, \pi)$ of a trajectory from step t until the end of the episode T under policy π:

$$Pr(a_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, \pi) = \pi(a_t \mid s_t) T(s_{t+1} \mid s_t, a_t) \pi(a_{t+1} \mid s_{t+1}) \ldots, T(s_T \mid s_{T-1}, a_{T-1})$$

$$= \prod_{k=t}^{T-1} \pi(a_k \mid s_k) T(s_{k+1} \mid s_k, a_k)$$

Importance sampling for off-policy Monte Carlo evaluation

Recall that our goal is to estimate $V^{\pi}(s) = \mathbb{E}\left[G_t \mid S_t = s, \pi\right] = \sum_{\tau} G_t Pr(a_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, \pi)$,

but we would like to use episodes $\tau^{(i)} = s_0^{(i)}, a_0^{(i)}, r_0^{(i)}, s_1^{(i)} \ldots s_{T^{(i)}}^{(i)}$ sampled under a different behavioral policy $b(a \mid s)$.

Consider the probability $Pr(a_t, r_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, \pi)$ of a trajectory from step t until the end of the episode T under policy π:

$$Pr(a_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, \pi) = \pi(a_t \mid s_t) T(s_{t+1} \mid s_t, a_t) \pi(a_{t+1} \mid s_{t+1}) \ldots, T(s_T \mid s_{T-1}, a_{T-1})$$

$$= \prod_{k=t}^{T-1} \pi(a_k \mid s_k) T(s_{k+1} \mid s_k, a_k)$$

Now, compute the likelihood ratio $\rho_{t:T-1}$ of the probabilities of this trajectory under π and under b.

$$\rho_{t:T-1} = \frac{Pr(a_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, \pi)}{Pr(a_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, b)} = \frac{\prod_{k=t}^{T-1} \pi(a_k \mid s_k) T(s_{k+1} \mid s_k, a_k)}{\prod_{k=t}^{T-1} b(a_k \mid s_k) T(s_{k+1} \mid s_k, a_k)}$$

Importance sampling for off-policy Monte Carlo evaluation

Recall that our goal is to estimate $V^{\pi}(s) = \mathbb{E}\left[G_t \mid S_t = s, \pi\right] = \sum_{\tau} G_t Pr(a_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, \pi)$,

but we would like to use episodes $\tau^{(i)} = s_0^{(i)}, a_0^{(i)}, r_0^{(i)}, s_1^{(i)} \ldots s_{T^{(i)}}^{(i)}$ sampled under a different behavioral policy $b(a \mid s)$.

Consider the probability $Pr(a_t, r_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, \pi)$ of a trajectory from step t until the end of the episode T under policy π:

$$Pr(a_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, \pi) = \pi(a_t \mid s_t) T(s_{t+1} \mid s_t, a_t) \pi(a_{t+1} \mid s_{t+1}) \ldots, T(s_T \mid s_{T-1}, a_{T-1})$$

$$= \prod_{k=t}^{T-1} \pi(a_k \mid s_k) T(s_{k+1} \mid s_k, a_k)$$

Now, compute the likelihood ratio $\rho_{t:T-1}$ of the probabilities of this trajectory under π and under b.

$$\rho_{t:T-1} = \frac{Pr(a_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, \pi)}{Pr(a_t, s_{t+1}, a_{t+1}, \ldots, s_T \mid S_t = s, b)} = \frac{\prod_{k=t}^{T-1} \pi(a_k \mid s_k) T(s_{k+1} \mid s_k, a_k)}{\prod_{k=t}^{T-1} b(a_k \mid s_k) T(s_{k+1} \mid s_k, a_k)}$$

$$\Longrightarrow \rho_{t:T-1} = \frac{\prod_{k=t}^{T-1} \pi(a_k \mid s_k)}{\prod_{k=t}^{T-1} b(a_k \mid s_k)}$$

Importance sampling for off-policy Monte Carlo evaluation

So, we can compute $V^{\pi}(s) = \mathbb{E}\left[G_t \mid S_t = s, \pi\right] = \mathbb{E}\left[\rho_{t:T-1} G_t \mid S_t = s, b\right]$ using episodes sampled from the behavioral policy $b(a \mid s)$, instead of the target policy π.

Importance sampling for off-policy Monte Carlo evaluation

sample rollouts under policy $b(a|s)$.

So, we can compute $V^\pi(s) = \mathbb{E}\left[G_t \mid S_t = s, \pi\right] = \mathbb{E}\left[\rho_{t:T-1} G_t \mid S_t = s, b\right]$ using episodes sampled from the behavioral policy $b(a \mid s)$, instead of the target policy π.

Given N sampled episodes under policy $b(a \mid s)$, define the set $\mathcal{T}(s)$ as the set of all timesteps and episode indices (i, t) such that state s was visited at time t in episode i. We can compute our estimate $\hat{V}^\pi(s)$ as:

Ordinary importance sampling

$$\hat{V}^\pi(s) = \frac{1}{|\mathcal{T}(s)|} \sum_{(i,t) \in \mathcal{T}(s)} \rho_{t:T^{(i)}-1} G_t$$

Importance sampling for off-policy Monte Carlo evaluation

So, we can compute $V^\pi(s) = \mathbb{E}\left[G_t \mid S_t = s, \pi\right] = \mathbb{E}\left[\rho_{t:T-1} G_t \mid S_t = s, b\right]$ using episodes sampled from the behavioral policy $b(a \mid s)$, instead of the target policy π.

Given N sampled episodes under policy $b(a \mid s)$, define the set $\mathcal{T}(s)$ as the set of all timesteps and episode indices (i, t) such that state s was visited at time t in episode i. We can compute our estimate $\hat{V}^\pi(s)$ as:

Ordinary importance sampling

$$\hat{V}^\pi(s) = \frac{1}{|\mathcal{T}(s)|} \sum_{(i,t) \in \mathcal{T}(s)} \rho_{t:T^{(i)}-1} G_t$$

Or

Weighted importance sampling

$$\hat{V}^\pi(s) = \frac{\sum_{(i,t) \in \mathcal{T}(s)} \rho_{t:T^{(i)}-1} G_t}{\sum_{(i,t) \in \mathcal{T}(s)} \rho_{t:T^{(i)}-1}}$$

imagine

$\pi'(a|s) = \arg\max_a Q^\pi(s,a)$

$b(a|s) =$ $\begin{cases} \pi'(a|s) & \text{w.p. } 1-\varepsilon \\ \text{any other action uniformly} & \text{w.p. } \varepsilon \end{cases}$

Off-policy Monte Carlo control

Off-policy MC prediction (policy evaluation) for estimating $Q \approx q_\pi$

Input: an arbitrary target policy π
Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
    $Q(s,a) \in \mathbb{R}$ (arbitrarily)
    $C(s,a) \leftarrow 0$

Loop forever (for each episode):
    $b \leftarrow$ any policy with coverage of π
    Generate an episode following b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
    $G \leftarrow 0$
    $W \leftarrow 1$
    Loop for each step of episode, $t = T-1, T-2, \ldots, 0$, while $W \neq 0$:
        $G \leftarrow \gamma G + R_{t+1}$
        $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
        $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$
        $W \leftarrow W \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$

Figure courtesy of Sutton & Barto (2018).

Off-policy MC control, for estimating $\pi \approx \pi_*$

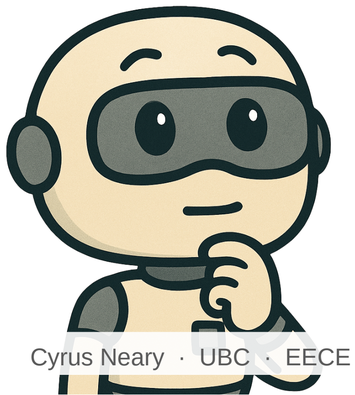
Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
    $Q(s,a) \in \mathbb{R}$ (arbitrarily)
    $C(s,a) \leftarrow 0$
    $\pi(s) \leftarrow \arg\max_a Q(s,a)$ (with ties broken consistently)

Loop forever (for each episode):
    $b \leftarrow$ any soft policy
    Generate an episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
    $G \leftarrow 0$
    $W \leftarrow 1$
    Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
        $G \leftarrow \gamma G + R_{t+1}$
        $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
        $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$
        $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken consistently)
        If $A_t \neq \pi(S_t)$ then exit inner Loop (proceed to next episode)
        $W \leftarrow W \frac{1}{b(A_t|S_t)}$

Figure courtesy of Sutton & Barto (2018).

Without an MDP model, why do we estimate $Q^\pi(s, a)$ rather than $V^\pi(s)$for control?

A: Q-values need fewer samples to estimate than state values.
B: MC cannot estimate V^π without a model.
C: Q-values always converge to optimal values, while V^π does not.
D: Greedy improvement from V^π needs models of R and T, while from Q^π it needs neither
E: I don't understand.



You run MC estimation of $Q^{\pi}(s, a)$ for a **deterministic** policy π with no exploring starts. What is the main problem?

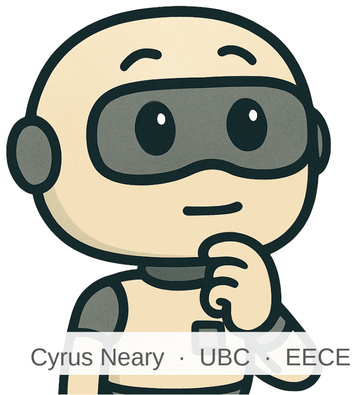
A: Only one action is ever taken in each state, so Q-values for the other actions are never estimated.

B: The returns will be biased estimates of Q^{π}.

C: Episodes are no longer guaranteed to terminate.

D: The policy improvement theorem no longer holds.

E: I don't understand.



You run on-policy first-visit MC control with a **fixed** $\varepsilon = 0.1$.
With infinitely many episodes, the policy converges to:

A: The optimal policy π^*.
B: The uniform random policy.
C: The best policy among ε-soft policies, which is generally not π^*.
D: Nothing because on-policy MC control can oscillate forever.
E: I don't understand.